Localo NestJS Backend Blueprint

Code standards, backend architecture, REST API catalogue, DTO plan, Prisma model mapping, module rules, and Codex implementation prompts for the complete Localo backend.

Prepared for	Businice Developers
Project	Localo
Version	v1.0 Backend Draft
Date	2026-06-04
Backend decision	Nx monorepo + NestJS + PostgreSQL + Prisma + REST APIs
Purpose	Give Codex a single implementation-ready backend spec so it does not waste tokens on repeated architecture decisions.

Decision: I am confident enough to create this v1 now. We do not need another discussion before drafting. Treat this PDF as the backend source of truth for review; after you review, we can update it before Codex starts implementation.

Companion document: Localo Complete Database Blueprint. This backend blueprint assumes that DB document is the database source of truth, and this file translates it into NestJS modules, APIs, DTOs, guards, services, and Codex build tickets.

Table of Contents

1. Backend Architecture Decision

Localo should be backend-first. The first implementation milestone is a stable API, database schema, seed data, auth, admin shop operations, and shop owner catalog management. UI work should consume these APIs after the backend is stable.

Layer	Decision	Reason
Monorepo	Nx	Matches Businice Developers architecture and supports apps plus shared packages.
API app	NestJS	Module-based backend architecture, guards, decorators, validation, services, testing, Swagger.
Language	TypeScript	Shared DTO/type safety across backend and frontend later.
Database	PostgreSQL	Strong relational data, JSONB, indexes, search-ready, analytics-ready.
ORM	Prisma	Clear schema, migrations, generated TypeScript client, Codex-friendly.
API style	REST first	Simpler for Admin, Shop Owner app, User app, Swagger, and Codex implementation.
Auth	JWT access token + refresh token sessions	Mobile and web friendly; supports logout and device sessions.
Validation	NestJS DTOs + class-validator	Consistent request validation and Swagger decorators.
Docs	Swagger/OpenAPI	Every endpoint should be visible and testable before UI starts.
Files	S3-compatible storage later; local/dev storage for MVP	Avoid blocking MVP on storage provider choice.
Queue/Cache	Not required for MVP; add Redis/BullMQ later	Keep first backend lean.

Core backend principles

- Backend owns all business rules. UI should not decide shop status, commission overdue state, permissions, invoice totals, or product visibility.

- Admin creates shops initially. Shop owner self-registration can be added later after approval and verification flows mature.

- Products and categories must be designed long-term, not only MVP. Category can be many-to-many and searchable through category links and tags.

- Commission billing must be ready from day one: monthly commission/invoice, payment status, overdue state, and admin pause/unpause controls.

- Location is a core feature, not an afterthought. API should support city/area search and shop latitude/longitude indexes.

2. Repository and Folder Structure

Use a clean Nx monorepo structure. Keep the API modular, and keep database code in a shared package so Prisma schema, migrations, and DB client are not hidden inside one app.

```
localo/
  apps/
    api/                          # NestJS backend application
      src/
        main.ts
        app.module.ts
        common/
          decorators/
          filters/
          guards/
          interceptors/
          pipes/
          responses/
          utils/
        config/
          env.schema.ts
          app.config.ts
          database.config.ts
          jwt.config.ts
        modules/
          auth/
          users/
          rbac/
          media/
          locations/
          shops/
          business-hours/
          categories/
```

```
        products/
        discovery/
        reviews/
        commerce/
        billing/
        support/
        notifications/
        admin/
        audit/
        settings/
        webhooks/
  admin/                      # Later: Admin UI
  mobile/                     # Later: User/mobile UI
  shop-owner/                 # Later: Shop owner UI

packages/
  db/                         # Prisma schema, migrations, seed, PrismaService
    prisma/
      schema.prisma
      migrations/
      seed.ts
    src/
      prisma.module.ts
      prisma.service.ts
      prisma-error.mapper.ts
  shared-types/               # Shared enums, API contracts, pagination types
  config/                     # Shared environment/config helpers
  utils/                      # Shared helpers only when truly reusable
```

Folder/file	Rule
*.module.ts	Imports controller, service, repository and module dependencies only.
*.controller.ts	HTTP layer only. No Prisma calls. No business logic. Uses guards, DTOs, Swagger.
*.service.ts	Business logic, transactions, permission-aware operations. Calls repository.
*.repository.ts	Prisma queries only. No HTTP errors unless mapped by service.
dto/*.dto.ts	Request DTOs, query DTOs, response DTOs where needed.
entities/*.entity.ts	Swagger response shape or domain response model. Do not duplicate Prisma models blindly.
guards/*.guard.ts	Module-specific guards only when common guards are not enough.
constants/*.ts	Statuses, allowed transitions, default values.
mappers/*.mapper.ts	Convert Prisma records to API response objects.

3. Global Backend Code Standards

Area	Standard
Naming	Files use kebab-case. Classes use PascalCase. Variables use camelCase. DB columns stay snake_case in Prisma using @map if needed.
Module names	Use plural business module names: users, shops, products, categories, support, billing.
Controllers	Controllers should be thin. Only route decorators, guards, DTO binding, and service calls.
Services	Services own business rules, state transitions, and transactions.
Repositories	Repositories own Prisma query composition and include/select definitions.
DTOs	Every POST/PATCH/PUT/query endpoint must have a DTO. No untyped body objects.
Validation	Use ValidationPipe globally with whitelist true, transform true, forbidNonWhitelisted true.
Responses	Use consistent response envelope for list, detail, mutation, and error responses.
Errors	Use domain-friendly exceptions. Never leak raw Prisma/database errors to client.
Pagination	Use page/perPage for admin lists; support cursor later for feeds/search if needed.
Soft delete	Business tables use deleted_at. Repository queries must exclude deleted records by default.
Audit	Admin/shop-owner sensitive actions must write audit_logs.
Transactions	Use Prisma transactions when creating shop + owner + address + commission settings, product + variants + media, invoice + items.
Security	Never return password_hash, refresh token hash, OTP, or internal metadata unless explicitly safe.
Swagger	Every controller must have ApiTags, ApiOperation, ApiResponse, ApiBearerAuth where protected.
Testing	Each core service gets unit tests for validation/business rules. Main endpoints get e2e smoke tests.

Global response shape

```
// Success detail
{
  "success": true,
  "message": "Shop created successfully",
  "data": { ... }
}

// Success list
{
  "success": true,
  "message": "Shops fetched successfully",
  "data": [ ... ],
  "meta": {
    "page": 1,
    "perPage": 20,
    "total": 124,
    "totalPages": 7
  }
}

// Error
{
  "success": false,
  "message": "Validation failed",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [ ... ]
  }
}
```

4. Database Model Mapping

Use the database blueprint as the table/column/index source of truth. The table below maps those tables to NestJS modules and Prisma models so Codex can implement the backend without repeatedly scanning or rediscovering the model plan.

Domain	Prisma models / DB tables	NestJS module
Identity & Access	User, UserSession, Role, Permission, RolePermission, UserRole	auth, users, rbac
Media	MediaAsset	media
Location	Country, State, City, Area, UserAddress	locations
Shop Core	Shop, ShopLocation, ShopBusinessHour, ShopSpecialHour, ShopUser, ShopDocument, ShopStatusHistory	shops, business-hours
Category & Catalog	Category, ShopCategoryLink, ProductBrand, Product, ProductCategoryLink, ProductVariant, ProductInventory, ProductMedia, ProductAttributeDefinition, ProductAttributeValue	categories, products
Discovery & Engagement	UserFavorite, RecentlyViewedItem, SearchQuery, Review, ReviewReply	discovery, reviews
Commerce-ready	Cart, CartItem, Order, OrderItem, OrderStatusHistory	commerce
Commission & Billing	CommissionPlan, ShopCommissionSetting, ShopCommissionInvoice, ShopInvoiceItem, ShopPayment	billing
Support	SupportTicket, SupportMessage, SupportAttachment	support
Notifications	Notification	notifications
System Operations	AuditLog, AppSetting, ImportJob, WebhookEvent	audit, settings, imports, webhooks

Model naming convention

```
// Prisma model names are PascalCase singular.
model Shop {
  id          String   @id @default(uuid()) @db.Uuid
  name        String   @db.VarChar(180)
  slug        String   @unique
  status      ShopStatus @default(PENDING)
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime @updatedAt @map("updated_at") @db.Timestamptz
  deletedAt   DateTime? @map("deleted_at") @db.Timestamptz

  @@map("shops")
  @@index([status])
  @@index([slug])
}
```

5. API Design Standards

Standard	Rule
Versioning	Prefix all routes with /api/v1. Future breaking changes use /api/v2.
Route style	Use resource-first REST routes. Avoid action-heavy routes except state transitions like approve, pause, mark-paid.
Admin routes	Use /admin/... for platform admin operations.
Shop owner routes	Use /owner/... for shop-owner and shop-staff operations.
Public routes	Use /public/... or unguarded routes only for user-facing discovery and catalog.
Auth routes	Use /auth/... only for login/session identity operations.
IDs	Use UUID in APIs for private/admin routes; public shop pages can use slug.
List filters	All list endpoints accept page, perPage, search, sortBy, sortOrder when useful.
Dates	Use ISO 8601 strings in requests/responses. Store as timestamptz.
Money	Use integer minor units or Decimal. Never use float for money.
Status updates	State transition endpoints must validate allowed transitions and write audit/history rows.

Pagination query DTO standard

```
export class PaginationQueryDto {
  @IsOptional() @Type(() => Number) @Min(1)
  page = 1;

  @IsOptional() @Type(() => Number) @Min(1) @Max(100)
  perPage = 20;

  @IsOptional() @IsString()
  search?: string;

  @IsOptional() @IsString()
  sortBy?: string;

  @IsOptional() @IsIn(['asc', 'desc'])
  sortOrder: 'asc' | 'desc' = 'desc';
}
```

6. Auth, RBAC, and Request Lifecycle

Localo must support multiple user types without duplicate accounts. One user can be customer, shop owner, staff, support agent, or admin based on roles and shop access. Guards must enforce platform role and shop-level permissions separately.

Layer	Responsibility
JwtAuthGuard	Validates access token and attaches user to request.
RolesGuard	Checks platform-level roles like super_admin, admin, support, customer.
PermissionsGuard	Checks permission codes such as shops.create, products.update, billing.mark_paid.
ShopAccessGuard	Checks that user has access to the requested shopId and role owner/manager/staff.
StatusGuard	Blocks suspended/deleted users and paused/suspended shops from restricted actions.

Recommended permission codes

- users.read
- users.update
- users.suspend
- shops.create
- shops.read
- shops.update
- shops.approve
- shops.pause
- shops.assign_owner
- categories.create
- categories.update

- categories.delete
- categories.reorder
- products.create
- products.read
- products.update
- products.delete
- products.publish
- reviews.moderate
- support.read
- support.reply
- billing.read
- billing.generate_invoice
- billing.mark_paid
- settings.update
- audit.read
- imports.run

7. Module-by-Module Backend Specification

Auth Module

Login, refresh tokens, logout, password reset, email/phone verification, and current-user session management.

Prisma models	User, UserSession
DTOs	LoginDto, RefreshTokenDto, ForgotPasswordDto, ResetPasswordDto, AuthUserResponseDto

Method	Path	Purpose	Guard
POST	/auth/login	Login with email or phone and password	Public
POST	/auth/refresh	Rotate refresh token and return new access token	Public + refresh token
POST	/auth/logout	Logout current session	JWT
POST	/auth/logout-all	Logout all user sessions	JWT
GET	/auth/me	Return current user profile, roles, permissions, shop access	JWT
POST	/auth/forgot-password	Start password reset	Public
POST	/auth/reset-password	Complete password reset	Public

- Hash passwords with a strong algorithm; never store raw tokens.
- Refresh token rotation must update UserSession.
- Suspended/deleted users cannot login.

Users and RBAC Modules

User profile, admin user management, roles, permissions, and role assignment.

Prisma models	User, Role, Permission, RolePermission, UserRole
DTOs	UpdateMeDto, UserQueryDto, UpdateUserStatusDto, AssignRolesDto

Method	Path	Purpose	Guard
GET	/users/me	Current user profile	JWT
PATCH	/users/me	Update current user profile	JWT
GET	/admin/users	Admin list users	Admin
GET	/admin/users/:id	Admin user detail	Admin

Method	Path	Purpose	Guard
PATCH	/admin/users/:id/status	Suspend/reactivate user	Admin
GET	/admin/roles	List roles	Admin
POST	/admin/users/:id/roles	Assign roles	Admin

- User status changes must write audit log.

- Do not let normal users assign roles or change user_type.

Media Module

Reusable uploaded media for avatars, shop logos, banners, products, documents, and support attachments.

Prisma models	MediaAsset
DTOs	CreateUploadDto, CompleteUploadDto, MediaResponseDto

Method	Path	Purpose	Guard
POST	/media/presign	Create upload target or local upload metadata	JWT
POST	/media/complete	Complete upload and create media asset	JWT
GET	/media/:id	Media metadata	JWT/Public depending visibility
DELETE	/media/:id	Soft delete media asset	Owner/Admin

- Do not accept arbitrary file types.

- Store owner_user_id and purpose for auditability.

- Actual binary upload strategy can be local/dev first, S3 later.

Locations and Addresses Module

Country/state/city/area catalog, geocoded search, and user/shop addresses.

Prisma models	Country, State, City, Area, UserAddress, ShopLocation
DTOs	LocationQueryDto, CreateUserAddressDto, UpdateUserAddressDto

Method	Path	Purpose	Guard
GET	/locations/countries	List countries	Public
GET	/locations/states	List states by country	Public
GET	/locations/cities	List cities by state/search	Public
GET	/locations/areas	List areas by city/search	Public
POST	/users/me/addresses	Create user address	JWT
GET	/users/me/addresses	List user addresses	JWT
PATCH	/users/me/addresses/:id	Update user address	JWT
DELETE	/users/me/addresses/:id	Delete user address	JWT

- Keep location master data seedable.

- For shop discovery, lat/lng and city/area indexes are mandatory.

Shops Module

Platform admin shop creation, approval, profile, ownership, status, location, and documents.

Prisma models	Shop, ShopLocation, ShopUser, ShopDocument, ShopStatusHistory, ShopCommissionSetting
DTOs	CreateShopDto, UpdateShopDto, UpdateShopStatusDto, AssignShopOwnerDto, OwnerUpdateShopProfileDto, ShopQueryDto

Method	Path	Purpose	Guard
POST	/admin/shops	Admin creates shop with owner and primary location	Admin + shops.create
GET	/admin/shops	Admin shop list with filters	Admin + shops.read
GET	/admin/shops/:id	Admin shop detail	Admin + shops.read
PATCH	/admin/shops/:id	Admin updates shop	Admin + shops.update

Method	Path	Purpose	Guard
PATCH	/admin/shops/:id/status	Approve/suspend/pause/reactivate shop	Admin + status permission
POST	/admin/shops/:id/owners	Assign shop owner	Admin + shops.assign_owner
GET	/owner/shops	Shop owner accessible shops	JWT
GET	/owner/shops/:shopId	Owner shop detail	ShopAccessGuard
PATCH	/owner/shops/:shopId/profile	Owner updates allowed profile fields	ShopAccessGuard
GET	/public/shops/:slug	Public shop page	Public

- Admin creates initial shops for MVP.

- Shop status transitions must write ShopStatusHistory and AuditLog.

- Paused shops should be hidden or restricted from product publishing based on business rule.

Business Hours Module

Weekly hours and special holiday/exception hours for shop discovery and open/closed UI.

Prisma models	ShopBusinessHour, ShopSpecialHour
DTOs	BusinessHourDto, ReplaceBusinessHoursDto, CreateSpecialHourDto, UpdateSpecialHourDto

Method	Path	Purpose	Guard
GET	/shops/:shopId/business-hours	Get weekly hours	ShopAccess/Public depending route
PUT	/owner/shops/:shopId/business-hours	Replace weekly hours	ShopAccessGuard
POST	/owner/shops/:shopId/special-hours	Create special hours	ShopAccessGuard
PATCH	/owner/shops/:shopId/special-hours/:id	Update special hours	ShopAccessGuard
DELETE	/owner/shops/:shopId/special-hours/:id	Delete special hours	ShopAccessGuard

- Validate no invalid time windows.

- Use backend utility to calculate is_open_now for public shop responses.

Categories Module

Hierarchical category system for shops, products, filters, and discovery.

Prisma models	Category, ShopCategoryLink, ProductCategoryLink
DTOs	CreateCategoryDto, UpdateCategoryDto, ReorderCategoriesDto, SetShopCategoriesDto, CategoryQueryDto

Method	Path	Purpose	Guard
GET	/public/categories	Public active category tree	Public
GET	/admin/categories	Admin category list/tree	Admin
POST	/admin/categories	Create category	Admin + categories.create
PATCH	/admin/categories/:id	Update category	Admin + categories.update
DELETE	/admin/categories/:id	Soft delete category	Admin + categories.delete
PATCH	/admin/categories/reorder	Reorder category tree	Admin + categories.reorder
PUT	/owner/shops/:shopId/categories	Set shop categories	ShopAccessGuard

- Support parent-child categories.

- Do not store category names only in products; use relation tables for filtering.

- Category can support tags/metadata for search.

Products and Catalog Module

Shop owner product/service catalog with categories, variants, price, discount, inventory, media, attributes, visibility, and publishing.

Prisma models	ProductBrand, Product, ProductCategoryLink, ProductVariant, ProductInventory, ProductMedia, ProductAttributeDefinition, ProductAttributeValue
DTOs	CreateProductDto, UpdateProductDto, ProductQueryDto, UpdateProductStatusDto, ProductVariantDto, ReplaceProductVariantsDto, ProductInventoryDto, AttachProductMediaDto

Method	Path	Purpose	Guard
GET	/owner/shops/:shopId/products	Owner product list	ShopAccessGuard
POST	/owner/shops/:shopId/products	Create product	ShopAccessGuard
GET	/owner/shops/:shopId/products/:id	Owner product detail	ShopAccessGuard
PATCH	/owner/shops/:shopId/products/:id	Update product	ShopAccessGuard
DELETE	/owner/shops/:shopId/products/:id	Soft delete product	ShopAccessGuard
PATCH	/owner/shops/:shopId/products/:id/status	Draft/publish/archive product	ShopAccessGuard
POST	/owner/shops/:shopId/products/:id/media	Attach product media	ShopAccessGuard
PUT	/owner/shops/:shopId/products/:id/variants	Replace variants	ShopAccessGuard
PATCH	/owner/shops/:shopId/products/:id/inventory	Update inventory	ShopAccessGuard
GET	/public/shops/:slug/products	Public shop products	Public
GET	/public/products/:id	Public product detail	Public

- Product create/update should be transactional.

- Price/discount validation must happen in service.

- Published products require active shop and required fields.

- Use categories + tags + search vector/index later for discovery.

Discovery, Favorites, Search, Reviews Module

Public browse/search, recently viewed, favorites, reviews, and review replies.

Prisma models	UserFavorite, RecentlyViewedItem, SearchQuery, Review, ReviewReply
DTOs	DiscoverShopQueryDto, GlobalSearchQueryDto, CreateFavoriteDto, CreateReviewDto, ReplyReviewDto, ModerateReviewDto

Method	Path	Purpose	Guard
GET	/public/discover/shops	Nearby/relevant shops	Public
GET	/public/search	Search shops/products/categories	Public
POST	/users/me/favorites	Favorite shop or product	JWT
GET	/users/me/favorites	List favorites	JWT
DELETE	/users/me/favorites/:id	Remove favorite	JWT
GET	/users/me/recently-viewed	Recently viewed	JWT
POST	/reviews	Create review	JWT
GET	/public/shops/:shopId/reviews	Public shop reviews	Public
POST	/owner/shops/:shopId/reviews/:id/reply	Shop reply to review	ShopAccessGuard
PATCH	/admin/reviews/:id/status	Moderate review	Admin

- Search queries should be logged for future product/category tuning.

- Only customers can review; backend must prevent duplicate/unfair reviews based on business rules.

Commerce-ready Module

Foundation for cart/order/enquiry flows even if full payment/order processing is not MVP day one.

Prisma models	Cart, CartItem, Order, OrderItem, OrderStatusHistory
DTOs	AddCartItemDto, UpdateCartItemDto, CreateOrderDto, OrderQueryDto, UpdateOrderStatusDto

Method	Path	Purpose	Guard
GET	/cart	Get current cart	JWT
POST	/cart/items	Add item to cart	JWT
PATCH	/cart/items/:id	Update cart item qty/options	JWT

Method	Path	Purpose	Guard
DELETE	/cart/items/:id	Remove cart item	JWT
POST	/orders	Create order/enquiry from cart	JWT
GET	/orders	Customer orders	JWT
GET	/owner/shops/:shopId/orders	Shop orders/enquiries	ShopAccessGuard
PATCH	/owner/shops/:shopId/orders/:id/status	Update order status	ShopAccessGuard

- Commerce can be feature-flagged if not used in MVP UI.

- Order status changes must write OrderStatusHistory.

Billing, Commission, and Payments Module

Commission plans, shop commission settings, invoices, invoice items, offline/manual payments, overdue pause logic.

Prisma models	CommissionPlan, ShopCommissionSetting, ShopCommissionInvoice, ShopInvoiceItem, ShopPayment
DTOs	CreateCommissionPlanDto, UpdateCommissionPlanDto, SetShopCommissionDto, InvoiceQueryDto, GenerateInvoiceDto, MarkInvoicePaidDto, RecordShopPaymentDto

Method	Path	Purpose	Guard
GET	/admin/commission-plans	List commission plans	Admin
POST	/admin/commission-plans	Create commission plan	Admin
PATCH	/admin/commission-plans/:id	Update commission plan	Admin
PUT	/admin/shops/:shopId/commission	Set shop commission config	Admin
GET	/admin/billing/invoices	Admin invoice list	Admin
POST	/admin/billing/invoices/generate	Generate invoice for period/shop	Admin
PATCH	/admin/billing/invoices/:id/mark-paid	Mark invoice paid	Admin
PATCH	/admin/billing/invoices/:id/mark-overdue	Mark invoice overdue	Admin
POST	/admin/shops/:shopId/pause-for-non-payment	Pause shop for unpaid commission	Admin
GET	/owner/shops/:shopId/billing/invoices	Owner invoice list	ShopAccessGuard
GET	/owner/shops/:shopId/billing/payments	Owner payment history	ShopAccessGuard

- Invoice totals must be generated by backend, never provided by UI.

- If invoice unpaid past grace period, backend/admin can pause shop.

- All billing changes need audit logs.

Support and Communication Module

Support tickets raised by shop owners/users, messages, attachments, assignment, status, priority.

Prisma models	SupportTicket, SupportMessage, SupportAttachment, MediaAsset
DTOs	CreateSupportTicketDto, SupportTicketQueryDto, CreateSupportMessageDto, UpdateSupportTicketDto

Method	Path	Purpose	Guard
POST	/support/tickets	Create support ticket	JWT
GET	/support/tickets	User's tickets	JWT
GET	/support/tickets/:id	Ticket detail	Ticket owner/Admin/Support
POST	/support/tickets/:id/messages	Add message	Ticket owner/Admin/Support
PATCH	/admin/support/tickets/:id	Assign, priority, status update	Admin/Support
GET	/admin/support/tickets	Admin support inbox	Admin/Support

- Ticket messages should preserve sender role.

- Attachments should reference MediaAsset instead of raw URLs.

Notifications Module

In-app notification records for admin, shop owner, staff, and users. Push/email can be added later.

Prisma models	Notification
DTOs	NotificationQueryDto, MarkNotificationReadDto, CreateNotificationDto

Method	Path	Purpose	Guard
GET	/notifications	List my notifications	JWT
PATCH	/notifications/:id/read	Mark one read	JWT
PATCH	/notifications/read-all	Mark all read	JWT
POST	/admin/notifications	Send admin notification	Admin

- Create notifications for shop approval, shop pause, invoice due/paid, support reply, product moderation later.

Admin, Settings, Audit, Imports, Webhooks Module

Platform operations: dashboard, app settings, audit logs, imports, and webhook idempotency.

Prisma models	AuditLog, AppSetting, ImportJob, WebhookEvent plus aggregate reads from all modules
DTOs	DashboardQueryDto, AuditLogQueryDto, UpdateAppSettingDto, CreateImportJobDto, WebhookPayloadDto

Method	Path	Purpose	Guard
GET	/admin/dashboard	Admin dashboard metrics	Admin
GET	/admin/audit-logs	Audit log search	Admin
GET	/admin/settings	List app settings	Admin
PATCH	/admin/settings/:key	Update app setting	Admin
POST	/admin/imports	Start import job	Admin
GET	/admin/imports	List import jobs	Admin
POST	/webhooks/:provider	Receive payment/app webhook	Public signed webhook

- Audit logs are append-only.

- Webhook events must be idempotent by provider event id.

- Settings updates must validate expected data type.

8. API Endpoint Master Catalogue

This catalogue is intentionally duplicated from module specs in compact form. Codex can implement one module at a time without re-reading the whole document.

Module	Method	Path	Guard
Auth	POST	/auth/login	Public
Auth	POST	/auth/refresh	Public + refresh token
Auth	POST	/auth/logout	JWT
Auth	POST	/auth/logout-all	JWT
Auth	GET	/auth/me	JWT
Auth	POST	/auth/forgot-password	Public
Auth	POST	/auth/reset-password	Public
Users and RBACs	GET	/users/me	JWT
Users and RBACs	PATCH	/users/me	JWT
Users and RBACs	GET	/admin/users	Admin
Users and RBACs	GET	/admin/users/:id	Admin
Users and RBACs	PATCH	/admin/users/:id/status	Admin
Users and RBACs	GET	/admin/roles	Admin
Users and RBACs	POST	/admin/users/:id/roles	Admin
Media	POST	/media/presign	JWT
Media	POST	/media/complete	JWT
Media	GET	/media/:id	JWT/Public depending visibility
Media	DELETE	/media/:id	Owner/Admin

Module	Method	Path	Guard
Locations and Addresses	GET	/locations/countries	Public
Locations and Addresses	GET	/locations/states	Public
Locations and Addresses	GET	/locations/cities	Public
Locations and Addresses	GET	/locations/areas	Public
Locations and Addresses	POST	/users/me/addresses	JWT
Locations and Addresses	GET	/users/me/addresses	JWT
Locations and Addresses	PATCH	/users/me/addresses/:id	JWT
Locations and Addresses	DELETE	/users/me/addresses/:id	JWT
Shops	POST	/admin/shops	Admin + shops.create
Shops	GET	/admin/shops	Admin + shops.read
Shops	GET	/admin/shops/:id	Admin + shops.read
Shops	PATCH	/admin/shops/:id	Admin + shops.update
Shops	PATCH	/admin/shops/:id/status	Admin + status permission
Shops	POST	/admin/shops/:id/owners	Admin + shops.assign_owner
Shops	GET	/owner/shops	JWT
Shops	GET	/owner/shops/:shopId	ShopAccessGuard
Shops	PATCH	/owner/shops/:shopId/profile	ShopAccessGuard
Shops	GET	/public/shops/:slug	Public
Business Hours	GET	/shops/:shopId/business-hours	ShopAccess/Public depending route
Business Hours	PUT	/owner/shops/:shopId/business-hours	ShopAccessGuard
Business Hours	POST	/owner/shops/:shopId/special-hours	ShopAccessGuard
Business Hours	PATCH	/owner/shops/:shopId/special-hours/:id	ShopAccessGuard
Business Hours	DELETE	/owner/shops/:shopId/special-hours/:id	ShopAccessGuard
Categories	GET	/public/categories	Public
Categories	GET	/admin/categories	Admin
Categories	POST	/admin/categories	Admin + categories.create
Categories	PATCH	/admin/categories/:id	Admin + categories.update
Categories	DELETE	/admin/categories/:id	Admin + categories.delete
Categories	PATCH	/admin/categories/reorder	Admin + categories.reorder
Categories	PUT	/owner/shops/:shopId/categories	ShopAccessGuard
Products and Catalog	GET	/owner/shops/:shopId/products	ShopAccessGuard
Products and Catalog	POST	/owner/shops/:shopId/products	ShopAccessGuard
Products and Catalog	GET	/owner/shops/:shopId/products/:id	ShopAccessGuard
Products and Catalog	PATCH	/owner/shops/:shopId/products/:id	ShopAccessGuard
Products and Catalog	DELETE	/owner/shops/:shopId/products/:id	ShopAccessGuard
Products and Catalog	PATCH	/owner/shops/:shopId/products/:id/status	ShopAccessGuard
Products and Catalog	POST	/owner/shops/:shopId/products/:id/media	ShopAccessGuard
Products and Catalog	PUT	/owner/shops/:shopId/products/:id/variants	ShopAccessGuard
Products and Catalog	PATCH	/owner/shops/:shopId/products/:id/inventory	ShopAccessGuard
Products and Catalog	GET	/public/shops/:slug/products	Public
Products and Catalog	GET	/public/products/:id	Public
Discovery, Favorites, Search, Reviews	GET	/public/discover/shops	Public
Discovery, Favorites, Search, Reviews	GET	/public/search	Public
Discovery, Favorites, Search, Reviews	POST	/users/me/favorites	JWT
Discovery, Favorites, Search, Reviews	GET	/users/me/favorites	JWT
Discovery, Favorites, Search, Reviews	DELETE	/users/me/favorites/:id	JWT

Module	Method	Path	Guard
Discovery, Favorites, Search, Reviews	GET	/users/me/recently-viewed	JWT
Discovery, Favorites, Search, Reviews	POST	/reviews	JWT
Discovery, Favorites, Search, Reviews	GET	/public/shops/:shopId/reviews	Public
Discovery, Favorites, Search, Reviews	POST	/owner/shops/:shopId/reviews/:id/reply	ShopAccessGuard
Discovery, Favorites, Search, Reviews	PATCH	/admin/reviews/:id/status	Admin
Commerce-ready	GET	/cart	JWT
Commerce-ready	POST	/cart/items	JWT
Commerce-ready	PATCH	/cart/items/:id	JWT
Commerce-ready	DELETE	/cart/items/:id	JWT
Commerce-ready	POST	/orders	JWT
Commerce-ready	GET	/orders	JWT
Commerce-ready	GET	/owner/shops/:shopId/orders	ShopAccessGuard
Commerce-ready	PATCH	/owner/shops/:shopId/orders/:id/status	ShopAccessGuard
Billing, Commission, and Payments	GET	/admin/commission-plans	Admin
Billing, Commission, and Payments	POST	/admin/commission-plans	Admin
Billing, Commission, and Payments	PATCH	/admin/commission-plans/:id	Admin
Billing, Commission, and Payments	PUT	/admin/shops/:shopId/commission	Admin
Billing, Commission, and Payments	GET	/admin/billing/invoices	Admin
Billing, Commission, and Payments	POST	/admin/billing/invoices/generate	Admin
Billing, Commission, and Payments	PATCH	/admin/billing/invoices/:id/mark-paid	Admin
Billing, Commission, and Payments	PATCH	/admin/billing/invoices/:id/mark-overdue	Admin
Billing, Commission, and Payments	POST	/admin/shops/:shopId/pause-for-non-payment	Admin
Billing, Commission, and Payments	GET	/owner/shops/:shopId/billing/invoices	ShopAccessGuard
Billing, Commission, and Payments	GET	/owner/shops/:shopId/billing/payments	ShopAccessGuard
Support and Communication	POST	/support/tickets	JWT
Support and Communication	GET	/support/tickets	JWT
Support and Communication	GET	/support/tickets/:id	Ticket owner/Admin/Support
Support and Communication	POST	/support/tickets/:id/messages	Ticket owner/Admin/Support
Support and Communication	PATCH	/admin/support/tickets/:id	Admin/Support
Support and Communication	GET	/admin/support/tickets	Admin/Support
Notifications	GET	/notifications	JWT
Notifications	PATCH	/notifications/:id/read	JWT
Notifications	PATCH	/notifications/read-all	JWT
Notifications	POST	/admin/notifications	Admin
Admin, Settings, Audit, Imports, Webhooks	GET	/admin/dashboard	Admin

Module	Method	Path	Guard
Admin, Settings, Audit, Imports, Webhooks	GET	/admin/audit-logs	Admin
Admin, Settings, Audit, Imports, Webhooks	GET	/admin/settings	Admin
Admin, Settings, Audit, Imports, Webhooks	PATCH	/admin/settings/:key	Admin
Admin, Settings, Audit, Imports, Webhooks	POST	/admin/imports	Admin
Admin, Settings, Audit, Imports, Webhooks	GET	/admin/imports	Admin
Admin, Settings, Audit, Imports, Webhooks	POST	/webhooks/:provider	Public signed webhook

9. DTO Master Catalogue

DTOs below are the minimum implementation targets. Codex should create DTO files even when DTOs feel small, because this keeps validation, Swagger, and frontend contracts predictable.

DTO	Fields / notes
LoginDto	identifier: email or phone; password
RefreshTokenDto	refreshToken
ForgotPasswordDto	identifier
ResetPasswordDto	token, newPassword
UpdateMeDto	fullName, avatarMediaId, metadata
UserQueryDto	page, perPage, search, userType, status, sortBy, sortOrder
UpdateUserStatusDto	status, reason
AssignRolesDto	roleIds[]
CreateUploadDto	filename, mimeType, sizeBytes, purpose, ownerType, ownerId?
CompleteUploadDto	uploadKey/path, checksum?, metadata
LocationQueryDto	countryId, stateId, cityId, search, isActive
CreateUserAddressDto	label, line1, line2, cityId, areaId, postalCode, latitude, longitude, isDefault
CreateShopDto	name, slug?, description, contactPhone, contactEmail, ownerUserId or owner payload, categoryIds[], location, commissionPlanId
UpdateShopDto	name, description, logoMediaId, bannerMediaId, contact fields, metadata
UpdateShopStatusDto	status, reason, effectiveAt?
AssignShopOwnerDto	userId, role, permissions?
OwnerUpdateShopProfileDto	description, contact fields, logoMediaId, bannerMediaId, socialLinks, metadata
ShopQueryDto	page, perPage, search, status, cityId, areaId, categoryId, ownerId
BusinessHourDto	dayOfWeek, opensAt, closesAt, isClosed
ReplaceBusinessHoursDto	hours: BusinessHourDto[]
CreateSpecialHourDto	date, opensAt, closesAt, isClosed, reason
CreateCategoryDto	name, slug?, parentId?, description, iconMediaId?, sortOrder, isActive, metadata
UpdateCategoryDto	name, slug, parentId, description, iconMediaId, sortOrder, isActive, metadata
ReorderCategoriesDto	items: [{id, parentId?, sortOrder}]
SetShopCategoriesDto	categoryIds[]
CreateProductDto	name, slug?, description, categoryIds[], brandId?, tags[], basePrice, discountType, discountValue, sku?, mediaIds[], variants?, attributes?, status
UpdateProductDto	same as create but partial
ProductQueryDto	page, perPage, search, categoryId, status, minPrice, maxPrice, hasDiscount, sortBy, sortOrder
UpdateProductStatusDto	status, reason?
ProductVariantDto	id?, name, sku, price, discount, optionValues, isPrimary, isActive
ProductInventoryDto	variantId?, quantity, lowStockThreshold, trackInventory

DTO	Fields / notes
AttachProductMediaDto	mediaId, sortOrder, isPrimary, altText
DiscoverShopQueryDto	lat, lng, radiusKm, cityId, areaId, categoryId, search, openNow, page, perPage
GlobalSearchQueryDto	q, type: all/shop/product/category, lat, lng, cityId, categoryId, page, perPage
CreateFavoriteDto	targetType: shop/product, targetId
CreateReviewDto	shopId, productId?, rating, title?, comment?, mediaIds[]
ReplyReviewDto	message
ModerateReviewDto	status, reason
AddCartItemDto	shopId, productId, variantId?, quantity, options?, notes?
UpdateCartItemDto	quantity, options?, notes?
CreateOrderDto	cartId?, shopId, items?, deliveryOrPickup, customerNotes, addressId?
UpdateOrderStatusDto	status, reason
CreateCommissionPlanDto	name, type, percentageRate?, fixedAmount?, billingCycle, graceDays, isActive
SetShopCommissionDto	commissionPlanId, customRate?, billingStartDate, isActive
GenerateInvoiceDto	shopId?, billingPeriodStart, billingPeriodEnd, dueDate
MarkInvoicePaidDto	paidAt, paymentMethod, referenceNo, notes
CreateSupportTicketDto	subject, description, category, priority, shopId?, attachments[]
CreateSupportMessageDto	message, attachments[]
UpdateSupportTicketDto	status, priority, assignedToId, internalNote?
NotificationQueryDto	page, perPage, isRead, type
CreateNotificationDto	recipientId, title, body, type, data, channel
AuditLogQueryDto	actorId, action, entityType, entityId, dateFrom, dateTo, page, perPage
UpdateAppSettingDto	value, metadata?
CreateImportJobDto	type, fileMediaId, options

10. Service, Repository, and Transaction Rules

Operation	Must be transactional because
Admin create shop	Creates shop, owner/shop_user relation, primary location, categories, commission setting, audit log.
Shop status update	Updates shop, inserts status history, may create notification, audit log.
Product create/update	Creates product, category links, variants, inventory, media, attribute values.
Invoice generation	Creates invoice, line items, totals, due date, notifications.
Mark invoice paid	Updates invoice, creates payment, may reactivate shop, audit log.
Order creation	Creates order, items, status history, cart state changes.
Support message with attachments	Creates message, attachment links, notification.

Repository rules

- Each repository should expose clear methods such as findById, findMany, create, update, softDelete, exists.

- Repository methods should accept Prisma.TransactionClient when used inside service transactions.

- Avoid massive include trees by default. Use select/include variants per endpoint response.

- All default queries for soft-deleted business tables must add deletedAt: null.

- Service should map Prisma errors to Conflict, NotFound, BadRequest, or Internal errors through a common error mapper.

11. Testing, Seeds, Migrations, and Observability

Area	Minimum requirement
Unit tests	AuthService, ShopsService, ProductsService, BillingService, SupportService.
E2E smoke	Login, create shop, approve shop, create product, public search, generate invoice, support ticket.

Area	Minimum requirement
Seeds	Roles, permissions, admin user, default app settings, base categories, commission plans, location examples.
Migrations	Prisma migrations must be committed. Never use db push for production.
Logging	Request id, method, path, user id, status code, duration. Avoid logging secrets.
Audit	Admin and billing state changes must write audit_logs.
Swagger	Generated automatically at /api/docs in dev/staging.
Health	GET /health should check API up and database connectivity.

```
Seed order:
1. roles
2. permissions
3. role_permissions
4. super admin user
5. app settings
6. countries/states/cities/areas sample
7. base categories
8. default commission plans
9. sample shop + owner + products for local development
```

12. Backend Build Order

Phase	What Codex should implement	Acceptance check
1	Nx API app, config, validation pipe, Swagger, health endpoint	npm/nx serve works; /api/docs and /health work.
2	Prisma package, schema from DB blueprint, migrations, seed roles/admin/categories/commission	Database migrates and seeds cleanly.
3	Auth + Users + RBAC	Admin login returns JWT; /auth/me returns roles/permissions.
4	Admin Shops + Locations + Business Hours	Admin creates shop with owner and location in one transaction.
5	Categories	Admin category tree and owner shop category mapping work.
6	Products/Catalog	Owner creates product with categories, variants, inventory, media.
7	Public discovery/search/favorites/reviews	Public shop/product endpoints and user engagement work.
8	Billing/commission	Generate invoice, mark paid/overdue, pause shop for non-payment.
9	Support/notifications/audit/settings	Support inbox, notifications, audit log search work.
10	Commerce-ready cart/order foundation	Feature-flagged APIs ready for future order UI.

13. Codex Implementation Prompt Pack

Use these prompts one at a time. This keeps Codex focused and prevents it from scanning or rewriting the full repository unnecessarily.

Prompt 1 - Backend foundation

```
You are working in the Localo Nx monorepo. Implement only the NestJS backend foundation.
Scope:
- Create apps/api NestJS structure if missing.
- Add global ValidationPipe with whitelist, transform, forbidNonWhitelisted.
- Add Swagger at /api/docs.
- Add config module with env validation.
- Add /health endpoint.
- Do not implement business modules yet.
- Run build/test or provide exact commands if not runnable.
Follow the Localo NestJS Backend Blueprint v1.0.
```

Prompt 2 - Prisma and database

```
Implement packages/db with Prisma for Localo.
Scope:
- Create Prisma schema using the Localo Complete Database Blueprint.
- Use PostgreSQL, UUIDs, timestamptz, soft delete fields, indexes, relations.
- Add PrismaService and PrismaModule.
- Add seed.ts for roles, permissions, admin user, categories, commission plans, app settings.
```

```
- Do not implement controllers yet.
- Keep naming consistent with Backend Blueprint.
```

Prompt 3 - Auth/users/RBAC

```
Implement Auth, Users, and RBAC modules only.
Scope:
- Login, refresh, logout, logout-all, /auth/me.
- Users current profile and admin user list/status update.
- Roles/permissions assignment basics.
- JWT guards, RolesGuard, PermissionsGuard decorators.
- DTOs, services, repositories, Swagger.
- No shop/product implementation in this task.
```

Prompt 4 - Admin shops

```
Implement Shops, Locations, and Business Hours modules for backend-first MVP.
Scope:
- Admin creates shop with owner and primary location transactionally.
- Admin shop list/detail/update/status/assign owner.
- Owner shop list/detail/profile update.
- Business hours replace and special hours CRUD.
- Write audit logs and shop_status_history for status changes.
- Use DTOs and guards from the Backend Blueprint.
```

Prompt 5 - Categories and products

```
Implement Categories and Products modules.
Scope:
- Admin category CRUD/tree/reorder.
- Owner set shop categories.
- Owner product CRUD with categories, variants, inventory, media, publish/archive status.
- Public shop products and product detail.
- Transactional create/update for product relations.
- Do not implement billing/support in this task.
```

Prompt 6 - Billing/support/notifications

```
Implement Billing, Support, Notifications, Audit, and Settings modules.
Scope:
- Commission plans, shop commission settings, invoices, mark paid/overdue, pause shop for non-payment.
- Support tickets/messages/attachments and admin inbox.
- Notifications list/read/create.
- Audit log search and app settings.
- Use existing Prisma models and guards.
```

14. Open Decisions Before Coding

Decision	Recommended default for v1
Payments for shop commission	Start with manual/offline record payment. Add Razorpay/Stripe webhook later.
Product type	Support product/service via product_type enum from day one.
Self registration	Disable shop owner self-registration in MVP; admin creates shops.
Location picker	Backend supports lat/lng/city/area now; UI library choice can come later.
Search engine	Use PostgreSQL indexes/search first. Add Meilisearch/Typesense later if needed.
File storage	Use local/dev adapter now; S3-compatible abstraction later.
Orders	Keep commerce-ready tables/APIs feature-flagged until UI/business flow is ready.
Tax/GST	Do not hardcode tax. Store billing tax config in app settings/commission plan metadata.

End of Backend Blueprint

After review, update this blueprint before giving it to Codex. The best workflow is: review PDF -> freeze decisions -> implement backend module by module -> build UI on top of stable APIs.